

Do toque no botão à tela redesenhada: o essencial da P1 de Flutter.

O fio condutor das duas apostilas é um ciclo: **interface** → **entrada** → **evento** → **processamento** → **estado** → **atualização da interface**. Os módulos 3 a 6 montam esse ciclo para um valor; os módulos 7 a 9 repetem o mesmo ciclo para uma coleção. Esta é a versão condensada: definições, tabelas, código-modelo e as respostas-chave dos 18 exercícios.

9	18	4	3
módulos + 1 prévia	exercícios do material	classes no esqueleto de todo app	fórmulas (°F, média, IMC)

Se o tempo for curto: esqueleto de 4 classes (M2) → setState com o contador (M3) → TextField + controller + dispose (M5) → tryParse ?? 0 e if/else (M6) → ListView.builder com Card/ListTile (M7-8) → cadastro dinâmico com add/removeAt/Expanded (M9). Escreva o Módulo 9 de cabeça: é o exercício "final" e o candidato mais forte a questão de código longa.

Sumário

MÓDULOS

1 · Flutter, Dart e o projeto	2
2 · Anatomia de um app	2
3 · Estado e setState()	3
4 · Layout: Row, Column e cia.	4
5 · Entrada: TextField e controller	5
6 · Conversão, decisão e fórmulas	5
7 · ListView e ListView.builder	6

8 · Card, ListTile e List<Produto>	7
9 · Lista dinâmica e Expanded	8
10 · Extra: navegação (prévia)	9

FERRAMENTAS

Colinha e scripts-modelo	9
Fórmulas e erros frequentes	12
Lacunas e o que não cai	12

■ Provável na prova ■ Código Dart/Flutter ■ Exemplo resolvido

1 · Flutter, Dart e a estrutura do projeto

O problema que o Flutter resolve: aplicações modernas precisam rodar em mais de um ambiente (Android, iOS, web, desktop). Com Flutter, **um único código gera o app para todos**.

Termo	O que é
Flutter	Framework + engine de interface: desenha a tela e oferece os widgets. Não é linguagem.
Dart	A linguagem; "faz toda a parte do desenvolvimento".
Widget	Todo elemento da tela: texto, botão, espaço, coluna, a própria tela.

Instalação (aula 2): instalar o Flutter (`C:\dev_flutter`) → colocar o `bin` no **Path** (para o terminal reconhecer `flutter`) → instalar o **Android Studio** (SDK do Android e emulador, mesmo programando no VS Code).

Item	Alterar?	Para que serve
<code>pubspec.yaml</code>	Sim	Configuração: nome, versão, SDK, dependências e declaração de assets . (Grafia: <code>pubspec</code> , de <i>pub specification</i> .)
<code>lib/</code>	Sim	Todo o código Dart; <code>lib/main.dart</code> é o ponto de entrada.
<code>assets/</code>	Sim	Imagens, fontes; precisa ser declarada no <code>pubspec.yaml</code> .
<code>android/</code> , <code>ios/</code> ...	Melhor não	Projetos nativos gerados por plataforma.
<code>build/</code> , <code>.dart_tool/</code>	Não	Gerados na compilação.
<code>test/</code>	Sim	Testes de widget.

Terminal

```
flutter create meu_app # cria o projeto
flutter pub get        # baixa as dependências do
pubspec.yaml
flutter run            # compila e abre no emulador
flutter doctor         # confere Flutter, SDK e Path
```

Onde eu mexo?

Mudar o texto do AppBar → `lib/main.dart` (ou a tela em `lib/`). Adicionar um logo → imagem em `assets/` + declarar no `pubspec.yaml`. Usar pacote externo → `dependencies:` do `pubspec.yaml` + `flutter pub get`.

Provável na prova

- "Qual problema o Flutter resolve?" → um código, vários ambientes. · "Qual a linguagem?" → Dart. · "Para que servem `pubspec.yaml`, `lib` e `assets`?"

2 · Anatomia de um app Flutter

Peça	Papel
<code>void main() + runApp()</code>	Ponto de entrada; <code>runApp</code> recebe o widget raiz e o coloca na tela.
<code>MeuApp extends StatelessWidget</code>	Retorna o MaterialApp : tema Material, <code>home</code> (primeira tela), <code>debugShowCheckedModeBanner: false</code> (tira a faixa DEBUG).
<code>Tela extends StatefulWidget</code>	"Casca" imutável; só cria o estado em <code>createState()</code> .
<code>_TelaState extends State<Tela></code>	Guarda as variáveis que mudam e o <code>build()</code> que desenha a tela.

Esqueleto completo (as 20 linhas de todo exercício)

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MeuApp()); // widget raiz
}

class MeuApp extends StatelessWidget {
  const MeuApp({super.key});
  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: TelaContador(), // primeira tela
    );
  }
}
```

	StatelessWidget	StatefulWidget
Dados que mudam?	Não	Sim (estado)
Classes	1 (com build)	2 (widget + State)
Redesenha com	—	setState()
Exemplos	MeuApp, TelaDisciplinas, TelaCursos, TelaProdutos (lista fixa)	TelaContador, TelaCadastro, Calculadora, TelaProdutos (cadastro dinâmico)

Scaffold = estrutura padrão de uma tela Material: `AppBar`: `AppBar(title: Text(...))` e `body`. **Detalhes:** `_` torna a classe privada ao arquivo; `@override` marca método sobrescrito; `const` = widget que nunca muda (não pode em `Text('$numero')`); `{super.key}` identifica o widget na árvore; `BuildContext` = posição do widget na árvore.

Classifique

Tela com 5 textos fixos → Stateless. Tela de IMC que mostra resultado após o clique → Stateful. Lista de 4 produtos escritos no código → Stateless (M2 Ex. 4). Cadastro onde o usuário adiciona produtos → Stateful (M2 Ex. 5).

Provável na prova

- "Diferencie Stateless e Stateful com exemplo." · "Complete o código" (`createState`, `extends State<...>`, `runApp`). · "Função do Scaffold / MaterialApp / `debugShowCheckedModeBanner`."

3 · Estado e setState()

Estado: qualquer dado que pode mudar enquanto a tela existe e aparece na interface (`int numero`, `String resultado`, `List<Produto> produtos`). Declarado na classe `_XState`, **fora do build** (dentro, voltaria ao valor inicial a cada redesenho).

setState(() { ... }): executa a alteração e avisa o Flutter que o `build()` precisa rodar de novo. Sem ele, a variável muda na memória mas a tela continua mostrando o valor antigo. Só existe em State (StatefulWidget).

toque → `onPressed` chama o método → `setState` altera a variável → `build()` roda de novo → `Text` mostra o novo valor

Regra de negócio no método: "a condição `if (numero > 0)` é verificada antes da redução; a regra 'o contador não pode ser negativo' fica protegida no próprio método". O `if` pode ficar dentro ou fora do `setState` (dentro, o `setState` roda sem mudança — inofensivo).

onPressed: diminuir (sem parênteses) passa a **referência** da função para o clique; `diminuir()` chamaria na hora do build. Com parâmetro: `() ⇒ removerProduto(index)`.

```
class TelaContador extends StatefulWidget {
  const TelaContador({super.key});
  @override
  State<TelaContador> createState() =>
    _TelaContadorState();
}

class _TelaContadorState extends State<TelaContador> {
  int numero = 0; // estado
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Contador')),
      body: Center(child: Text('$numero')),
    );
  }
}
```

Ex. 1 (contador) — mesmo padrão do Ex. 8 (estoque)

```
int numero = 0;

void aumentar() {
  setState(() {
    numero++; // muda e pede redesenho
  });
}

void diminuir() {
  if (numero > 0) { // regra: nunca negativo
    setState(() {
      numero--;
    });
  }
}

// no build: referência ao método, SEM parênteses
ElevatedButton(onPressed: diminuir, child: const Text('-'));
```

Ex. 8 "Zerar com confirmação": segunda variável de estado `bool confirmandoZerar`; o 1º clique liga a flag e troca o texto do botão (`Text(confirmandoZerar ? 'Confirmar' : 'Zerar estoque')`); o 2º zera; Entrada/Saída cancela. Estado controlando a própria interface.

Rastreie o contador

Início 0; toques -, +, -, -, -: - → 0 (bloqueado) · + → 1 · + → 2 · - → 1 · - → 0 · - → 0 (bloqueado). Final: 0; setState rodou 4 vezes (nos bloqueados nem é chamado). Outro: +, -, -, + → 1, 0, 0, 1 → final 1.

Provável na prova

- Escrever contador/estoque com + e - que nunca fica negativo. · "O que acontece se remover o setState()?" · Rastrear o valor final após uma sequência de cliques.

4 · Layout: Column, Row e widgets de espaço

Widget	Faz	Parâmetros
Column	Filhos na vertical	children, mainAxisAlignment, crossAxisAlignment
Row	Filhos na horizontal	children, mainAxisAlignment
Center	Centraliza o filho	child
Padding	Espaço interno em volta (afasta das bordas)	padding: EdgeInsets.all(20)
SizedBox	Espaço fixo entre widgets (no eixo principal)	height: 30 (Column) · width: 20 (Row)
Text	Texto	style: TextStyle(fontSize, fontWeight), textAlign
ElevatedButton	Botão	onPressed, child
Divider	Linha separadora	—

	Eixo principal (mainAxisAlignment)	Eixo cruzado (crossAxisAlignment)
Column	vertical	horizontal
Row	horizontal	vertical

center agrupa no meio (Ex. 1, 8) · **spaceEvenly** espaços iguais entre e nas pontas (4 botões da calculadora, Ex. 4) ·

spaceBetween encosta o 1º e o último nas bordas ·

CrossAxisAlignment.stretch estica na largura. **child** recebe um widget; **children** recebe uma lista.

Árvore de widgets do Ex. 1

```
Scaffold
├─ appBar: AppBar(title: Text('Contador'))
└─ body: Center
    └─ Column (mainAxisAlignment: center)
        ├─ Text('Valor atual')  fontSize 20
        ├─ Text('$numero')    fontSize 50, bold
        ├─ SizedBox(height: 30)
        └─ Row (mainAxisAlignment: center)
            ├─ ElevatedButton('-')
            └─ SizedBox(width: 20)
                └─ ElevatedButton('+')
```

Provável na prova

- Dado um desenho de tela, escrever a árvore Column/Row/SizedBox. · "Qual MainAxisAlignment distribui 4 botões com espaços iguais?" → spaceEvenly. · "mainAxisAlignment em Row × em Column."

5 · Entrada de dados: TextField e TextEditingController

"Cada TextField possui um controller. O conteúdo digitado é obtido com `controller.text`. Como os controllers permanecem ativos enquanto a tela existe, devem ser liberados em `dispose()`, antes de `super.dispose()`." (Ex. 2) – o trio **TextField + controller + dispose()** é cobrado junto.

Elemento	Função
<code>TextEditingController()</code>	Objeto que guarda o texto do campo (<code>final</code> : o objeto não muda, o conteúdo sim)
<code>controller: nomeController</code>	Liga o campo ao controller
<code>nomeController.text</code>	Lê o texto digitado (sempre String)
<code>.clear()</code> · <code>.text.trim()</code>	Apaga o campo · remove espaços nas pontas
<code>keyboardType: TextInputType.number</code>	Abre o teclado numérico (não impede letras)
<code>InputDecoration(labelText: , border: OutlineInputBorder())</code>	Rótulo e borda
<code>dispose()</code>	Libera os controllers quando a tela é destruída; sem ele, vazamento de memória. Primeiro os seus, super.dispose() por último

Quantos controllers?

Ex. 2 (nome, idade) 2 · Ex. 4 (dois números) 2 · Ex. 5 (Celsius) 1 · Ex. 6 (três notas) 3 · Ex. 7 (peso, altura) 2 · cadastro de produtos 2. Regra: **1 controller por TextField, e cada um aparece no dispose()**.

Ex. 2 (cadastro)

```
final TextEditingController nomeController =
    TextEditingController();
final TextEditingController idadeController =
    TextEditingController();
String mensagem = '';

void mostrarDados() {
    String nome = nomeController.text; // lê o
    campo
    String idade = idadeController.text;
    setState(() {
        mensagem = 'Olá, $nome! Você tem $idade anos.';
    });
}

@override
void dispose() {
    nomeController.dispose(); // 1º os
    controllers
    idadeController.dispose();
    super.dispose(); // por
    último
}

// no build
TextField(
    controller: idadeController,
    keyboardType: TextInputType.number,
    decoration: const InputDecoration(
        labelText: 'Idade', border: OutlineInputBorder()),
),
```

Provável na prova

- "Para que serve o TextEditingController e por que chamar dispose()?" · Completar o dispose() de uma tela com 3 campos. · Achar o erro: `super.dispose()` antes dos controllers, ou controller sem `dispose`.

6 · Conversão, decisão e fórmulas

Exercícios 3 a 7 são o mesmo padrão: **entrada → conversão → decisão → atualização da interface**.

"`int.tryParse(idadeController.text) ?? 0` tenta converter o texto para int e utiliza 0 caso a conversão não seja possível." `??` = operador "se nulo". `tryParse` nunca lança erro; `int.parse('abc')` lançaria `FormatException` e travaria o app. `int` para

contagens e idades; **double** para notas, peso, altura, temperatura, preço.

Expressão	Resultado	Por quê
<code>int.tryParse('18')</code>	18	inteiro válido
<code>int.tryParse('abc') ?? 0</code>	0	null → ?? troca por 0
<code>int.tryParse('12.5')</code>	null	int não aceita decimal
<code>int.tryParse('')</code>	null	campo vazio
<code>double.tryParse('1.80')</code>	1.8	ponto é o separador
<code>double.tryParse('1,80')</code>	null	vírgula não é aceita; altura "1,80" vira 0
<code>double.tryParse(' 7 ')</code>	7.0	espaços nas pontas ignorados

Exercício	Decisão
3 · Maioridade	<code>idade >= 18</code> → maior; senão menor
4 · Calculadora	<code>n2 != 0</code> → divide; senão "Não é possível dividir por zero." (7 / 0 em double dá Infinity, não erro: validação explícita)
6 · Média	<code>media >= 6</code> → Aprovado; senão Reprovado
7 · IMC	<code>altura > 0</code> ; depois <code>< 18.5</code> abaixo · <code>< 25</code> adequado · senão sobrepeso

Saída: `'$nome'` interpola variável; `'${n1 + n2}'` expressão (chaves); `valor.toStringAsFixed(2)` → "24.69"; `'R\$ ...'` (\$) literal com barra; `\n` quebra linha.

Provável na prova

- Escrever o método de cálculo dos Ex. 5–7 ou variante (km → milhas, desconto de 10%). · `int.tryParse(texto) ?? 0` para 'abc'? e '12.5'? · "Por que a divisão exige validação?" · Calcular à mão (86 °F, 7.0, 24.69).

Ex. 4 – métodos auxiliares + validação

```
double lerNumero1() {
  return double.tryParse(numero1Controller.text) ?? 0;
}

void dividir() {
  double n1 = lerNumero1();
  double n2 = lerNumero2();
  setState(() {
    if (n2 != 0) {
      resultado = 'Resultado: ${n1 / n2}';
    } else {
      resultado = 'Não é possível dividir por zero.';
    }
  });
}
```

O método auxiliar evita repetir a conversão nas 4 operações. O cálculo pode ficar fora do `setState`; o que importa é a variável exibida mudar dentro dele.

Valores da apostila

Ex. 5: $C = 30 \rightarrow F = 30 \times 9/5 + 32 = 54 + 32 = 86^{\circ}F$ (`toStringAsFixed(0)` para bater "86"). Ex. 6: $(7+8+6)/3 = 7.0 \rightarrow$ **Aprovado**; $(5+7+5)/3 = 5.67 \rightarrow$ Reprovado. Ex. 7: $80 / (1.80 \times 1.80) = 80 / 3.24 = 24.69 \rightarrow$ **Peso adequado**; $57 / 1.62^2 = 21.72$. Ex. 3: idade "dezoito" → null → 0 → "é menor de idade".

7 · ListView e ListView.builder

Ideia central do professor: uma coleção Dart guarda dados; um `ListView` apresenta widgets; o trabalho do app é transformar cada elemento da coleção no widget correspondente. Modelo mental: lista de dados → index → `itemBuilder` → widget.

	Column	ListView(children)	ListView.builder
Rolagem	não (estoura a tela)	sim	sim
Itens	escritos à mão	escritos à mão	gerados a partir dos dados

M2 Ex. 2

```
final List<String> cursos = ['Flutter', 'Python', 'Java', 'Banco de Dados'];

return Scaffold(
  appBar: AppBar(title: const Text('Cursos')),
  body: ListView.builder(
    itemCount: cursos.length, // quantos
    itens
    itemBuilder: (context, index) { // chamado para
      cada posição
      return ListTile(
```

Bom para	formulários curtos	poucos itens fixos	listas vindas de List, tamanho variável; só constrói os visíveis
Apostila	Ex. 1-8	M2 Ex. 1 (Disciplinas, com Divider)	M2 Ex. 2 a 5

Peça	Função
itemCount	Quantos itens podem ser construídos (<code>cursos.length</code>). Maior que a lista → <code>RangeError</code> .
itemBuilder: (context, index) { ... }	Transforma cada posição da coleção em um widget.
index	Posição atual: 0, 1, 2... até <code>length - 1</code> .

`List<String>` = lista cujos elementos são textos; o tipo entre `< >` diz o que a lista guarda.

Provável na prova

- "Column × ListView" e "ListView × ListView.builder". · "Explique itemCount, itemBuilder e index." · Fixação 1: `List<String>` com 8 linguagens em builder + `ListTile` (colinha).

```
leading: const Icon(Icons.school),
title: Text(cursos[index]), // dado da
posição index
);
},
),
);
```

O que o builder produz?

4 cursos → `length 4`, index 0 a 3: Flutter, Python, Java, Banco de Dados. `subtitle: Text('Curso ${index + 1}')` mostra "Curso 1" a "Curso 4". 8 cursos → último `index = 7`.

8 · Card, ListTile e List<Produto>

Posição do ListTile	Papel	Exemplo
leading	início	<code>Icon(Icons.computer, size: 35)</code>
title	informação principal	<code>Text(produto.nome)</code> em negrito
subtitle	complementar	<code>'R\$ 120.00 • Estoque: 5'</code>
trailing	extremidade final	<code>Icon(arrow_forward_ios)</code> ou <code>IconButton</code> lixeira

Card "agrupa visualmente informações relacionadas" (sombra, borda arredondada); `margin:`

`EdgeInsets.only(bottom: 10)` separa os Cards; aceita qualquer child (`ListTile` só dá a estrutura pronta). No Material 2 o Card é só visual; `onTap` vem no próximo material.

De String para objeto: "uma lista de String armazena um texto por posição; um item real tem vários atributos" — a lista "passa a representar entidades do domínio" (orientação a objetos). `required this.nome` = parâmetro nomeado obrigatório que já atribui ao atributo; parâmetros nomeados (entre chaves) deixam a criação legível e sem ordem fixa; atributos `final` : para "alterar", cria-se outro objeto. `Produto` não é widget; é uma classe Dart de dados.

M2 Ex. 4

```
class Produto {
  final String nome;
  final double preco;
  final int quantidade;
  Produto({ // parâmetros
    nomeados
    required this.nome, // obrigatório
    required this.preco,
    required this.quantidade,
  });
}

final List<Produto> produtos = [
  Produto(nome: 'Teclado', preco: 120.00, quantidade: 5),
  Produto(nome: 'Mouse', preco: 80.00, quantidade: 10),
];

// dentro do itemBuilder
final Produto produto = produtos[index];
return Card(
  child: ListTile(
    leading: const Icon(Icons.inventory_2, size: 35),
    title: Text(produto.nome,
      style: const TextStyle(fontWeight:
        FontWeight.bold)),
    subtitle: Text('R$
      ${produto.preco.toStringAsFixed(2)} '
        '• Estoque: ${produto.quantidade}'),
    trailing: const Icon(Icons.arrow_forward_ios),
  ),
);
```

3º Card?

Teclado (120, 5), Mouse (80, 10), Monitor (950, 3), Headset (250, 7) → index 2 → **Monitor · R\$ 950.00 · Estoque: 3.** length = 4. Valor total do estoque: 600 + 800 + 2850 + 1750 = 6000.

Provável na prova

- "As quatro posições do ListTile." · Fixação 2 e 3: classe Livro (título, autor) ou Aluno (nome, nota) em Cards (colinha). · "Por que List<Produto> em vez de List<String>?"

9 · Lista dinâmica: add, removeAt, isEmpty e Expanded

O exercício que "reúne os conteúdos anteriores com estado e entrada de dados": **controller + objeto + add/removeAt + setState + Expanded** numa tela só.

Elemento	Função
<code>final List<Produto> produtos = [];</code>	A lista como estado: começa vazia. <code>final</code> impede trocar a lista, não mudar o conteúdo.
<code>add(obj)</code> · <code>removeAt(index)</code>	Adiciona ao final · remove pela posição (o builder já tem o index; <code>remove(obj)</code> procura o objeto)
<code>isEmpty</code> · <code>length</code>	Está vazia? · quantos elementos
<code>cond ? A : B</code>	Ternário: mensagem ou lista. "Uma interface precisa prever o que mostrar quando ainda não há dados."
Expanded	Entrega ao filho o espaço restante dentro de Row/Column
<code>IconButton(icon:, onPressed:)</code>	Botão só com ícone (lixeira no trailing)

Adicionar: TextField → controller → Produto → add() → setState() → builder → Card
Excluir: IconButton → removeAt(index) → setState() → Card desaparece

Por que Expanded? "Uma ListView vertical dentro de uma Column precisa receber uma altura limitada. Expanded faz a lista ocupar o espaço restante e evita o conflito de altura ilimitada." Sem ele: *Vertical viewport was given unbounded height*.

`onPressed: () { removerProduto(index); }` — a função anônima "guarda" o index daquele Card. Os `clear()` ficam fora do setState: o controller avisa o próprio TextField.

M2 Ex. 5 — partes essenciais

```
final List<Produto> produtos = [];  
  
void adicionarProduto() {  
  final String nome = nomeController.text.trim();  
  final double preco =  
    double.tryParse(precoController.text) ?? 0;  
  if (nome.isEmpty || preco <= 0) { // validação: sai  
    return;  
  }  
  setState(() {  
    produtos.add(Produto(nome: nome, preco: preco));  
  });  
  nomeController.clear(); // limpa os campos  
  precoController.clear();  
}  
  
void removerProduto(int index) {  
  setState(() { produtos.removeAt(index); });  
}  
  
// no build, último filho da Column  
Expanded(  
  child: produtos.isEmpty  
    ? const Center(child: Text('Nenhum produto cadastrado.'))  
    : ListView.builder(  
      itemCount: produtos.length,  
      itemBuilder: (context, index) {  
        final Produto produto = produtos[index];  
        return Card(  
          child: ListTile(  
            leading: const Icon(Icons.inventory_2),  
            title: Text(produto.nome),  
            subtitle: Text('R\$  
            ${produto.preco.toStringAsFixed(2)}'),  
            trailing: IconButton(  
              icon: const Icon(Icons.delete),  
              onPressed: () { removerProduto(index);  
            },  
          ),  
        ),  
      ),  
    ),  
  ),  
);
```

Rastreie a lista

add("Mouse","80") → [Mouse] · add("", "50") → nome vazio, return · add("Teclado","abc") → preco 0, return · add("Monitor","950") → [Mouse, Monitor] · excluir o 1º → removeAt(0) → **[Monitor]**: 1 Card "Monitor – R\$ 950.00", length 1, isEmpty false.

Provável na prova

- Escrever o cadastro dinâmico (produtos, tarefas ou alunos) com add, removeAt, isEmpty e Expanded (Fixação 4 e 5). · "Por que Expanded no ListView dentro da Column?" · Rastrear o estado da lista após uma sequência de ações.

10 · Extra: navegação entre telas (prévia)

A apostila diz que **onTap**, **SnackBar** e

Navigator.push() / pop() são o próximo material – provavelmente **não caem na P1**. Mas o seu main.dart usa Navigator.push no menu dos exercícios. **Pilha de telas:** push coloca uma tela por cima; Navigator.pop(context) tira e volta (a seta do AppBar faz o pop). import ... show TelaContador; importa só aquela classe, evitando conflito entre os vários MeuApp/main.

Provável na prova

Baixa. Se cair: "o que faz Navigator.push?" → abre uma nova tela empilhando-a sobre a atual.

Seu main.dart

```
void abrir(BuildContext context, Widget tela) {  
  Navigator.push(  
    context,  
    MaterialPageRoute(builder: (context) => tela),  
  );  
}  
  
ElevatedButton(  
  onPressed: () => abrir(context, const TelaContador()),  
  child: const Text('Exercicio 1 – Contador'),  
),
```

Colinha e scripts-modelo

Grupo	Widget / recurso	Responde a	Grupo	Widget / recurso	Responde a
Estrutura	MaterialApp(home:), Scaffold(appBar:, body:), AppBar(title:)	Primeira tela e tema · estrutura da tela · barra superior	Listas	ListView(children:), ListView.builder(itemCount:, itemBuilder:)	Itens à mão · itens gerados de uma List
Layout	Column / Row · Center, Padding · SizedBox · Expanded · Divider	Empilhar · centralizar/afastar · espaço · ocupar o restante · linha		Card + ListTile	Item com leading/title/subtitle/trailing
Interação	ElevatedButton(onPressed:, child:), IconButton(icon:, onPressed:), TextField(controller:, keyboardType:, decoration:)	Botão com texto · só ícone · campo de entrada	Dart	int/double.tryParse(t) ?? 0 · '\$var' / '\${expr}' · x.toStringAsFixed(n) · add / removeAt / length / isEmpty · cond ? a : b	Converter com padrão · interpolar · casas decimais · manipular a lista · escolher

Modelo: campo, botão e resultado (Ex. 3, 5, 6, 7)

```
class _TelaXState extends State<TelaX> {  
  final TextEditingController valorController = TextEditingController();  
  String resultado = '';  
  
  void calcular() {  
    double v = double.tryParse(valorController.text) ?? 0; // 1. entrada  
    double r = v * 2; // 2. troque a fórmula
```

```

    setState(() {
        // 3. decisão + estado
        resultado = r ≥ 10 ? 'Alto: ${r.toStringAsFixed(2)}' : 'Baixo';
    });
}

@override
void dispose() { valorController.dispose(); super.dispose(); }

@override
Widget build(BuildContext context) {
    return Scaffold(
        appBar: AppBar(title: const Text('Tela X')),
        body: Padding(
            padding: const EdgeInsets.all(20),
            child: Column(children: [
                TextField(
                    controller: valorController,
                    keyboardType: TextInputType.number,
                    decoration: const InputDecoration(
                        labelText: 'Valor', border: OutlineInputBorder()),
                ),
                const SizedBox(height: 20),
                ElevatedButton(onPressed: calcular, child: const Text('Calcular')),
                const SizedBox(height: 30),
                Text(resultado, style: const TextStyle(fontSize: 20)),
            ]),
        ),
    );
}
}

```

Fixação 2 e 3 – Livro / Aluno em Cards

```

class Livro {
    final String titulo;
    final String autor;
    Livro({required this.titulo, required this.autor});
}

final List<Livro> livros = [
    Livro(titulo: 'Dom Casmurro', autor: 'Machado de Assis'),
    Livro(titulo: 'Vidas Secas', autor: 'Graciliano Ramos'),
];
// itemBuilder
final Livro livro = livros[index];
return Card(child: ListTile(
    leading: const Icon(Icons.book),
    title: Text(livro.titulo), subtitle: Text(livro.autor)));

// Aluno: class Aluno { final String nome; final double nota; ... }
// subtitle: Text('Nota: ${aluno.nota.toStringAsFixed(1)}')

```

Fixação 1 – 8 linguagens (Stateless + builder)

```

class TelaLinguagens extends StatelessWidget {
    const TelaLinguagens({super.key});
    @override
    Widget build(BuildContext context) {
        final List<String> linguagens = ['Dart', 'Python', 'Java', 'C',
            'C#', 'JavaScript', 'Kotlin', 'Swift'];
        return Scaffold(
            appBar: AppBar(title: const Text('Linguagens')),
            body: ListView.builder(
                itemCount: linguagens.length,
                itemBuilder: (context, index) => ListTile(
                    leading: const Icon(Icons.code),
                    title: Text(linguagens[index]),
                ),
            ),
        );
    }
}

```

```
}  
}
```

Fixação 4 – tarefas (add + removeAt)

```
final TextEditingController tarefaController = TextEditingController();  
final List<String> tarefas = [];  
  
void adicionar() {  
  final String t = tarefaController.text.trim();  
  if (t.isEmpty) return;  
  setState(() { tarefas.add(t); });  
  tarefaController.clear();  
}  
// build: Column [ TextField, SizedBox, ElevatedButton(adicionar),  
Expanded(  
  child: tarefas.isEmpty  
    ? const Center(child: Text('Nenhuma tarefa.'))  
    : ListView.builder(  
      itemCount: tarefas.length,  
      itemBuilder: (context, index) => Card(child: ListTile(  
        title: Text(tarefas[index]),  
        trailing: IconButton(  
          icon: const Icon(Icons.delete),  
          onPressed: () { setState(() { tarefas.removeAt(index); }); },  
        ),  
      )),  
    ),  
  ),  
),
```

Fixação 5 (estoque com nome, preço e quantidade) = o Módulo 9 com um terceiro campo/controller `quantidadeController`, `int.tryParse( ...) ?? 0`, e quantidade na classe e no subtitle.

Fórmulas e erros frequentes

Exercício	Fórmula	Exemplo
5 · Temperatura	$F = (C \times 9 / 5) + 32$	$30 \rightarrow 54 + 32 = 86 \text{ °F}$
6 · Média	$(n1 + n2 + n3) / 3 \geq 6$ aprova	$(7+8+6)/3 = 7.0 \rightarrow$ Aprovado
7 · IMC	$\text{peso} / (\text{altura} \times \text{altura})$	$80 / 3.24 = 24.69$
7 · Classificação	< 18.5 abaixo · < 25 adequado · senão sobrepeso	$24.69 \rightarrow$ Peso adequado
4 · Calculadora	$+ - \times \div (n2 \neq 0)$	$10 / 4 = 2.5$; $10 / 0 \rightarrow$ mensagem
3 · Maioridade	$\text{idade} \geq 18$	$18 \rightarrow$ maior; $17 \rightarrow$ menor
Listas	último index = length - 1 · exibido = index + 1	8 cursos $\rightarrow$ index 0..7, "Curso 1".. "Curso 8"

IMC passo a passo

Peso 80, altura 1.80: $1.80 \times 1.80 = 3.24 \rightarrow 80 / 3.24 = 24.691...$
 $\rightarrow \text{toStringAsFixed}(2) = "24.69" \rightarrow 18.5 \leq 24.69 < 25 \rightarrow$
"Peso adequado".

Sintoma	Causa $\rightarrow$ correção
Clico e a tela não muda	Alterou a variável sem setState $\rightarrow$ envolva em <code>setState(() { ... })</code>
Valor volta ao inicial	Variável dentro do build $\rightarrow$ declare na classe _XState
<i>Vertical viewport was given unbounded height</i>	ListView dentro de Column $\rightarrow$ envolva com Expanded
RangeError (index)	itemCount maior que a lista $\rightarrow$ <code>itemCount: lista.length</code>
Erro de const em Text	<code>const Text('\$numero')</code> com variável $\rightarrow$ tire o const
Altura "1,80" dá errado	<code>tryParse('1,80') = null</code> $\rightarrow$ 0 $\rightarrow$ digite com ponto ou <code>.replaceAll(',', '.')</code>
Idade "17.5" vira 0	<code>int.tryParse</code> não aceita decimal $\rightarrow$ use double
"R" some / erro de interpolação	<code>'R\$...'</code> sem barra $\rightarrow$ <code>'R\\$...'</code>
Método roda ao abrir a tela	<code>onPressed: calcular()</code> $\rightarrow$ <code>onPressed: calcular</code>
flutter test falha no meu_app	<code>widget_test.dart</code> usa MyApp do template $\rightarrow$ trocar para MeuApp ou apagar

Lacunas e o que não cai

- **Sem ementa nem cronograma:** o recorte foi feito pelo material dado (aulas 2 e 3 + duas apostilas). Confirme com o professor se é isto que cai e se o Material 2 (listas) entra.
- **Provavelmente fora da P1** (a apostila diz "pertencem ao próximo material"): onTap no ListTile, SnackBar, Navigator.push()/pop(), múltiplas telas, Form e TextFormField. O Módulo 10 é só prévia.
- **Aula 1** sem anotação (provável apresentação). Capa dos Exercícios de Continuação diz 27/08 e o cabeçalho 21/08 — mesmo conteúdo.
- Os blocos "Provável na prova" são palpites a partir dos exercícios; não há prova antiga no material. Pergunte se a prova é prática (no computador) ou escrita.